IDBP Phase 1 — Full Codebase Documentation

S.N. Polymers | Integrated Digital Business Platform

> **Scope:** This document is a complete file-by-file technical reference for every piece of code delivered in Phase 1. It is intended for developer onboarding, code reviews, and future Phase 2 handoff.

Table of Contents

---

1. Repository Layout

```
SNPolymers/                          ← Monorepo root
│
├── backend/                         ← Node.js + Express REST API
│   ├── src/
│   │   ├── app.js                   ← Express server entry point
│   │   ├── db/
│   │   │   └── supabase.js          ← Supabase client singleton
│   │   ├── routes/
│   │   │   ├── auth.routes.js       ← Public + authenticated auth endpoints
│   │   │   └── admin.routes.js      ← Admin-only CRUD endpoints
│   │   ├── controllers/
│   │   │   ├── auth.controller.js   ← OTP, verify, logout, /me handlers
│   │   │   └── admin.controller.js  ← User whitelist + sessions CRUD
│   │   ├── services/
│   │   │   ├── otp.service.js       ← OTP generation, hashing, verification
│   │   │   ├── whatsapp.service.js  ← Twilio WABA message delivery
│   │   │   ├── session.service.js   ← JWT issuance + session DB CRUD
│   │   │   └── email.service.js     ← Admin login/logout email alerts
│   │   └── middleware/
│   │       ├── verifyJwt.js         ← JWT cookie guard (all protected routes)
│   │       ├── requireAdmin.js      ← Role guard (admin-only routes)
│   │       └── rateLimiter.js       ← express-rate-limit configs
│   ├── .env                         ← Local secrets (gitignored)
│   ├── .env.example                 ← Template with all required keys
│   └── package.json
│
├── frontend/                        ← React 19 + Vite SPA
│   ├── public/assets/logo.png       ← Company logo asset
│   ├── index.html                   ← Vite HTML shell
│   ├── vite.config.js               ← Vite build configuration
│   ├── tailwind.config.js           ← Custom design token palette
│   ├── postcss.config.js            ← PostCSS pipeline
│   └── src/
│       ├── main.jsx                 ← React root mount
│       ├── App.jsx                  ← Route tree definition
│       ├── index.css                ← Tailwind directives
│       ├── api/
│       │   └── authApi.js           ← Axios instance (credentials + base URL)
│       ├── components/
│       │   ├── AuthContext.jsx       ← Global auth state provider
│       │   └── ProtectedRoute.jsx   ← Route guards (role-aware)
│       └── pages/
│           ├── Home.jsx             ← Public landing / corporate gateway
│           ├── Login.jsx            ← Mobile number entry + OTP request
│           ├── OtpVerify.jsx        ← 6-digit OTP input + timer
│           ├── Dashboard.jsx        ← Post-login console (staff + admin)
│           └── admin/
│               ├── AdminPanel.jsx   ← Whitelist management CRUD UI
│               └── AuditLog.jsx     ← Session history with filters
│
├── CODEBASE_DOCUMENTATION.md        ← This file
├── PHASE1_DOCUMENTATION.md          ← High-level phase summary
├── implementation_plan.md           ← Original design blueprint
├── README.md                        ← Setup & deployment guide
└── .gitignore
```

---

2. Technology Stack & Dependencies

Backend (`backend/package.json`)

Package	Version	Purpose

Package	Version	Purpose
`express`	^4.19.2	HTTP server and routing framework
`@supabase/supabase-js`	^2.43.4	Supabase DB client (PostgreSQL via REST)
`jsonwebtoken`	^9.0.2	JWT signing and verification (HS256)
`bcrypt`	^6.0.0	OTP hash generation and comparison
`uuid`	^14.0.0	Unique session JTI generation (`v4`)
`cookie-parser`	^1.4.6	Parses incoming `httpOnly` token cookies
`cors`	^2.8.5	Cross-Origin Resource Sharing headers
`helmet`	^8.2.0	Security-hardening HTTP headers
`dotenv`	^16.4.5	Environment variable loading from `.env`
`express-rate-limit`	^7.3.1	IP/mobile-level request throttling
`twilio`	^5.1.0	WhatsApp Business API (WABA) OTP delivery
`nodemailer`	^8.0.10	Gmail SMTP email alert transport
`nodemon` *(dev)*	^3.1.2	Hot-reload development server

npm scripts:

```
npm run dev    # nodemon src/app.js  (development with hot-reload)
npm run start  # node src/app.js     (production)
```

Frontend (`frontend/package.json`)

Package	Version	Purpose
`react`	^19.2.6	UI component library
`react-dom`	^19.2.6	React DOM renderer
`react-router-dom`	^6.23.1	Client-side routing (v6 data router API)
`axios`	^1.7.2	HTTP client with `withCredentials` support
`tailwindcss`	^3.4.4	Utility-first CSS framework
`vite`	^8.0.12	Development server and production bundler
`@vitejs/plugin-react`	^6.0.1	JSX transform + Fast Refresh
`postcss` + `autoprefixer`	^8.4 / ^10.4	CSS build pipeline

npm scripts:

```
npm run dev      # Vite dev server on :5173
npm run build    # Production bundle output to dist/
npm run preview  # Serve production build locally
npm run lint     # ESLint check
```

---

3. Database Schema (Supabase/PostgreSQL)

Three tables are defined in Supabase. All are protected by **Row Level Security (RLS)**. The backend accesses them exclusively via the **Service Role Key**, bypassing RLS for all server-side operations.

Table: `authorised_users`

Stores the mobile-number whitelist of personnel permitted to log in.

```
CREATE TABLE authorised_users (
  id            uuid         PRIMARY KEY DEFAULT gen_random_uuid(),
  mobile_number varchar(15)  UNIQUE NOT NULL,  -- E.164 format: +919876543210
  display_name  varchar(100),
  role          varchar(50)  DEFAULT 'staff',  -- 'staff' | 'admin'
  permissions   jsonb        DEFAULT '{}',      -- reserved for Phase 2 modules
  created_at    timestamptz  DEFAULT now(),
  is_active     boolean      DEFAULT true
);
```

Key behaviors:

- `is_active = false` blocks login even if the mobile number matches.
- `role = 'admin'` grants access to `/admin/*` routes.
- `permissions` (JSONB) is reserved for future module-level access flags (e.g., `{"manufacturing": true}`).

Table: `otp_requests`

Stores one bcrypt-hashed OTP record per send event. Never stores plaintext.

```
CREATE TABLE otp_requests (
  id            uuid         PRIMARY KEY DEFAULT gen_random_uuid(),
  mobile_number varchar(15)  NOT NULL,
  otp_hash      text         NOT NULL,     -- bcrypt hash, NOT the raw code
  expires_at    timestamptz  NOT NULL,     -- now() + 5 minutes
  is_used       boolean      DEFAULT false,
  attempts      int          DEFAULT 0,   -- incremented on wrong guess
  created_at    timestamptz  DEFAULT now()
);
```

Key behaviors:

- Only the **latest** record with `is_used = false` for a given mobile is evaluated.
- If `attempts >= 3`, the code is dead (locked) — user must request a new OTP.
- If `now() > expires_at`, the code is expired — user must request a new OTP.
- On success, `is_used` is set to `true` before the session is created.

Table: `sessions`

Complete audit ledger. One row per login event. Updated on logout.

```
CREATE TABLE sessions (
  id               uuid         PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          uuid         REFERENCES authorised_users(id),
  jwt_jti          varchar(100) UNIQUE,         -- UUID v4, embedded in JWT
  ip_address       varchar(45),                 -- supports IPv4 and IPv6
  user_agent       text,
  login_at         timestamptz  DEFAULT now(),
  logout_at        timestamptz,                 -- null while session is active
  duration_seconds int,                         -- computed on logout
  module           varchar(50)  DEFAULT 'office',
  is_active        boolean      DEFAULT true
);
```

Key behaviors:

- `jwt_jti` is the linkage between the JWT token and this DB record.

- On every protected API call, `verifyJwt` checks `is_active = true` on this row.
- Logout sets `is_active = false`, `logout_at`, and `duration_seconds`.
- Admin deactivation / deletion instantly sets `is_active = false` on all matching rows.

---

4. Backend — `backend/`

4.1 Entry Point — `app.js`

Responsibility: Express application factory. Registers all global middleware and mounts route handlers.

What it does:

Step	Code Section	Detail
1	Production sanity guards	If `NODE_ENV=production`, crashes on missing/default `JWT_SECRET` or localhost `FRONTEND_URL` — preventing accidental insecure deploys.
2	`helmet()`	Adds secure HTTP headers (X-Frame-Options, Content-Security-Policy, etc.).
3	`cors(corsOptions)`	Whitelists only the configured `FRONTEND_URL` origin. `credentials: true` required for cookie transport.
4	`express.json()`	Parses JSON request bodies.
5	`cookieParser()`	Makes `req.cookies.token` available to all middleware.
6	`app.set('trust proxy', 1)`	Enables accurate IP detection behind Vercel/Render reverse proxies (needed for rate limiting).
7	Route mounting	`/api/v1/auth` → `auth.routes.js`, `/api/v1/auth/admin` → `admin.routes.js`
8	`GET /health`	Simple liveness check endpoint for monitoring.
9	404 handler	Returns `{ success: false, message: 'Resource not found.' }`
10	Global error handler	Catches unhandled errors, logs stack trace, returns 500.

4.2 Database Client — `db/supabase.js`

Responsibility: Creates and exports a single shared Supabase client instance using the **Service Role Key**, which has full database access and bypasses all RLS policies.

```
// Key behavior
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY  // ← NOT the anon key
);
module.exports = { supabase };
```

> **IMPORTANT:** Using the Service Role Key means all database access is authenticated with full admin privileges. This is intentional — the Express API is the security boundary. Never expose this key to the client.

Startup warning: If env vars are missing, logs a `WARNING` but does not crash, so the dev server can still boot (with non-functional DB calls).

4.3 Routes

`routes/auth.routes.js`

Mounts at `/api/v1/auth`. Two public routes, two JWT-protected routes.

```
// Public — no token required
POST /request-otp   ← otpRequestLimiter → requestOtp
POST /verify-otp    ← otpVerifyLimiter  → verifyOtpCode

// Protected — requires valid JWT cookie
POST /logout        ← verifyJwt → logout
GET  /me            ← verifyJwt → getMe
```

`routes/admin.routes.js`

Mounts at `/api/v1/auth/admin`. **All routes** pass through `verifyJwt` then `requireAdmin` before reaching handlers.

```
router.use(verifyJwt);      // ← applied globally to the entire admin router
router.use(requireAdmin);   // ← role check after token verification

GET    /users        → getUsers
POST   /users        → addUser
PATCH  /users/:id    → updateUser
DELETE /users/:id    → removeUser
GET    /sessions     → getSessions
```

4.4 Controllers

`controllers/auth.controller.js`

Exports four functions:

`requestOtp(req, res)`

```
POST /api/v1/auth/request-otp
Body: { mobileNumber: string }
```

Flow:

1. Validates `mobileNumber` is present and matches E.164 regex.
2. Queries `authorised_users` where `mobile_number = $1 AND is_active = true`.
3. If not found → `403 Access denied`.
4. Generates 6-digit OTP via `generateOtp()`.
5. Hashes it via `hashOtp()` using bcrypt.
6. Stores hash + expiry via `storeOtp()`.
7. Delivers OTP via `sendOtp()` (WhatsApp or console log in dev).
8. Returns `200 { success: true }`.

`verifyOtpCode(req, res)`

```
POST /api/v1/auth/verify-otp
Body: { mobileNumber: string, otp: string }
```

Flow:

1. Calls `verifyOtp(mobileNumber, otp)` — returns `{ success, reason, attemptsLeft }`.
2. If failed → returns reason and remaining attempts.
3. Re-fetches user from `authorised_users`.
4. Calls `generateToken(user)` → `{ token, jti }`.
5. Calls `createSession({ userId, jti, ipAddress, userAgent })`.
6. Sets `res.cookie('token', token, { httpOnly: true, maxAge: 24h })`.
7. Fires `notifyAdminLogin(...)` asynchronously (non-blocking).
8. Returns `200 { success: true, user: { id, mobile_number, display_name, role, permissions } }`.

Cookie settings:

Environment	`httpOnly`	`secure`	`sameSite`

Environment	httpOnly	secure	sameSite
Development	✗	✗	lax
Production	✓	✓	none

`logout(req, res)`

```
POST /api/v1/auth/logout
Requires: Valid JWT cookie (via verifyJwt middleware)
```

Flow:

1. Gets `jti` from `req.jti` (set by `verifyJwt`).
2. Calls `closeSession(jti)` → updates DB, returns closed session row.
3. Calls `formatDuration(session.duration_seconds)` for human-readable duration.
4. Calls `res.clearCookie('token', ...)`.
5. Fires `notifyAdminLogout(...)` asynchronously.
6. Returns `200 { success: true }`.

`getMe(req, res)`

```
GET /api/v1/auth/me
Requires: Valid JWT cookie (via verifyJwt middleware)
```

Simply returns the `req.user` object attached by `verifyJwt`:

```
{
  "success": true,
  "user": {
    "id": "uuid",
    "mobile_number": "+91XXXXXXXXXX",
    "role": "staff|admin",
    "permissions": {},
    "displayName": "John Doe"
  }
}
```

`controllers/admin.controller.js`

Exports five functions. All require Admin JWT (enforced at route level).

`getUsers(req, res)` — `GET /admin/users`

- Fetches all rows from `authorised_users` ordered by `created_at DESC`.
- Also fetches all `sessions` rows (`user_id`, `login_at`).
- Merges to compute `session_count` and `last_login_at` per user.
- Returns enriched user list.

`addUser(req, res)` — `POST /admin/users`

```
Body: { mobileNumber, displayName?, role?, permissions? }
```

- Validates E.164 mobile number format.
- Inserts into `authorised_users`.
- Handles PostgreSQL error code `23505` (UNIQUE violation) → `409 Conflict`.

`updateUser(req, res)` — `PATCH /admin/users/:id`

```
Body: { displayName?, role?, permissions?, isActive? }
```

- Builds a partial update object from provided fields.
- If `isActive === false`: also closes all active sessions for this user immediately.

`removeUser(req, res)` — `DELETE /admin/users/:id`

1. Sets `is_active = false` + `logout_at = now()` on all active sessions for user.
2. Hard-deletes the user from `authorised_users`.

`getSessions(req, res)` — `GET /admin/sessions`

```
Query: ?userId=uuid&dateFrom=YYYY-MM-DD&dateTo=YYYY-MM-DD
```

- Performs a joined Supabase query (sessions + authorised_users) via `select('*, authorised_users(mobile_number, display_name, role)')`.
- Supports optional filters on `user_id`, `login_at >=`, `login_at <=`.
- Returns full session history ordered by `login_at DESC`.

4.5 Services

`services/otp.service.js`

Export	Signature	Description
`generateOtp`	`() → string`	`crypto.randomInt(100000, 999999).toString()` — cryptographically secure 6-digit code.
`hashOtp`	`(otp: string) → Promise<string>`	`bcrypt.hash(otp, 10)` — 10 salt rounds.
`storeOtp`	`(mobile, hash) → Promise<row>`	Inserts into `otp_requests` with `expires_at = now() + 5 min`.
`verifyOtp`	`(mobile, rawOtp) → Promise<result>`	Full verification pipeline (see below).

`verifyOtp` detailed logic:

```
1. SELECT latest is_used=false record for mobile, ORDER BY created_at DESC LIMIT 1
2. If none → { success: false, reason: 'No active OTP request found.' }
3. If now() > expires_at → { success: false, reason: 'OTP has expired.' }
4. If attempts >= 3 → { success: false, reason: 'Too many failed attempts...' }
5. bcrypt.compare(rawOtp, otpRequest.otp_hash)
   - If false → UPDATE attempts++ → { success: false, reason: 'Invalid OTP code.', attemptsLeft: 2 - attempts }
   - If true  → UPDATE is_used=true → { success: true }
```

`services/whatsapp.service.js`

Export	Signature	Description
`sendOtp`	`(mobileNumber, otp) → Promise<result>`	Sends OTP via Twilio WABA or falls back to console log.

Initialization logic:

```
// Only instantiates Twilio client if BOTH credentials are present and non-placeholder
if (accountSid && authToken &&
    accountSid !== 'your_twilio_account_sid' &&
    authToken   !== 'your_twilio_auth_token') {
  client = twilio(accountSid, authToken);
}
```

Fallback (development mode):

```
═══════════════════════════════════
[DEV WhatsApp Send] To: +91XXXXXXXXXX
[OTP CODE]: 847291
═══════════════════════════════════
```

Production message body:

```
Your Integrated Digital Business Platform (IDBP) security OTP code is: {otp}.
It is valid for 5 minutes. Do not share this code.
```

`services/session.service.js`

Export	Signature	Description
`generateToken`	`(user) → { token, jti }`	Signs JWT with `HS256`, embeds `jti = uuidv4()`.
`createSession`	`({ userId, jti, ipAddress, userAgent }) → Promise<row>`	Inserts session row in `sessions` table.
`closeSession`	`(jti) → Promise<row>`	Computes `duration_seconds`, updates `logout_at`, sets `is_active = false`.
`formatDuration`	`(seconds) → string`	Converts integer seconds to `HH:MM:SS` format string.

JWT payload structure:

```
{
  "user_id":      "uuid",
  "mobile_number": "+91XXXXXXXXXX",
  "role":         "staff | admin",
  "permissions":  {},
  "jti":          "uuid-v4",
  "iat":          1234567890,
  "exp":          1234567890
}
```

Duration formula:

```
Math.max(0, Math.floor((logout_at - login_at) / 1000))  // in seconds
```

`services/email.service.js`

Export	Signature	Description
`notifyAdminLogin`	`({ mobileNumber, displayName, role, ipAddress, userAgent })`	Fires login HTML email alert to `ADMIN_EMAIL`.
`notifyAdminLogout`	`({ mobileNumber, displayName, durationFormatted, logoutTime })`	Fires logout HTML email alert to `ADMIN_EMAIL`.

Transport initialization:

```
// Only creates real transport if Gmail credentials exist
if (gmailUser && gmailAppPassword) {
  transporter = nodemailer.createTransport({ service: 'gmail', auth: { user, pass } });
} else {
  // Falls back to console log (dev mode)
}
```

Non-blocking execution: Both functions call the internal `sendEmail()` helper using `transporter.sendMail(options, callback)` — the callback-based API means errors are logged but never throw, and the HTTP response is never delayed.

4.6 Middleware

`middleware/verifyJwt.js`

Applied to: `POST /logout`, `GET /me`, and all `/admin/*` routes.

Verification pipeline:

```
1. Extract token from req.cookies.token
   → If missing: 401 "Authentication required. No token provided."

2. jwt.verify(token, JWT_SECRET)
   → If invalid/expired: 401 "Authentication failed. Invalid or expired token."

3. SELECT is_active FROM sessions WHERE jwt_jti = decoded.jti LIMIT 1
   → If not found or is_active=false: clear cookie → 401 "Session is inactive or has been logged out."

4. SELECT is_active FROM authorised_users WHERE id = decoded.user_id LIMIT 1
   → If not found or is_active=false: 403 "Access denied. Account is deactivated or removed."

5. Attach to req:
   req.user = { id, mobile_number, role, permissions, displayName }
   req.jti  = decoded.jti

6. next()
```

> This double-check (JWT signature + live DB session status) ensures that a valid token issued to a deactivated or deleted user is still rejected in real time.

`middleware/requireAdmin.js`

Applied to: all routes under `admin.routes.js` (after `verifyJwt`).

```
if (!req.user || req.user.role !== 'admin') {
  return res.status(403).json({ success: false, message: 'Access denied. Administrator privileges required.' });
}
next();
```

`middleware/rateLimiter.js`

Two separate limiter instances exported:

Limiter	Applied to	Window	Max	Key
`otpRequestLimiter`	`POST /request-otp`	15 min	3 requests	`req.body.mobileNumber` or IP
`otpVerifyLimiter`	`POST /verify-otp`	5 min	5 requests	`req.body.mobileNumber` or IP

Key generator prefers mobile number over IP address, which means rate limiting follows the user's identity, not the network address. This prevents shared-IP environments (offices, NAT) from blocking innocent users.

Standard RateLimit headers (`RateLimit-*`) are returned in responses. Legacy `X-RateLimit-*` headers are disabled.

4.7 Environment Configuration — `.env.example`

```
# Server
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173

# Supabase
SUPABASE_URL=your_supabase_project_url
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key

# JWT
JWT_SECRET=generate_a_secure_at_least_256_bit_secret_string
JWT_EXPIRY=24h

# Twilio WhatsApp
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886

# Email Alerts
GMAIL_USER=your_gmail_sender@gmail.com
GMAIL_APP_PASSWORD=your_gmail_app_password
ADMIN_EMAIL=admin_notifications_receiver@domain.com
```

> **CAUTION:** The actual `.env` file is gitignored. Never commit real secrets. In production, inject these via Render's environment variable dashboard.

---

5. Frontend — `frontend/`

5.1 Build Configuration — `vite.config.js`

```
export default defineConfig({
  plugins: [react()],  // JSX transform + Fast Refresh
  server: {
    port: 5173,
    host: true           // Binds to 0.0.0.0 (accessible on LAN)
  }
})
```

SPA Routing on Vercel: The `vercel.json` in the frontend root is configured to rewrite all routes to `index.html`, enabling React Router to handle client-side navigation.

5.2 Design System — `tailwind.config.js`

The theme extends Tailwind with a custom **"Admin/Corporate" dark palette** and typography:

Token	Value	Usage
`admin-bg`	`#0b111e`	Page background — deep navy
`admin-card`	`#151c2c`	Card/container surfaces
`admin-border`	`#222f47`	Dividers, input borders
`admin-primary`	`#1d4ed8`	Royal blue accents
		Amber/gold (secondary CTA,

Token	Value	Usage
`admin-accent`	`#b45309`	badges)
`slate-950`	`#0b0f19`	Deepest input backgrounds
Font	`Inter`	Google Fonts system sans-serif

The amber/gold accent color (`amber-*` Tailwind scale) is used as the primary call-to-action color throughout all pages (buttons, highlights, badges).

5.3 Application Root — `main.jsx` & `App.jsx`

`main.jsx` mounts `<App />` into the `#root` DOM node using `ReactDOM.createRoot` .

`App.jsx` defines the full route tree, wrapped in `<AuthProvider>` :

```
<AuthProvider>
  <BrowserRouter>
    <Routes>
      /                   → <Home />                    (public)
      /login              → <Login />                   (public)
      /verify-otp         → <OtpVerify />               (public)

      <ProtectedRoute allowedRoles={['staff', 'admin']}>
        /dashboard        → <Dashboard />

      <ProtectedRoute allowedRoles={['admin']}>
        /admin            → <AdminPanel />
        /admin/sessions   → <AuditLog />

      *                   → Navigate to /               (404 fallback)
    </Routes>
  </BrowserRouter>
</AuthProvider>
```

The nested route pattern uses React Router v6's `<Outlet />` mechanism — `ProtectedRoute` renders its children only when auth conditions are met, otherwise redirects.

5.4 API Client — `api/authApi.js`

```
const authApi = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api/v1/auth',
  withCredentials: true,   // ← critical: sends the httpOnly token cookie
  headers: { 'Content-Type': 'application/json' }
});
```

`withCredentials: true` is mandatory for cross-origin cookie transport (Vite dev server on `:5173` → Express on `:5000`). This also applies in production (Vercel → Render).

Usage pattern across all pages:

```
// GET request
const { data } = await authApi.get('/me');

// POST request
const { data } = await authApi.post('/request-otp', { mobileNumber });

// Admin CRUD
await authApi.get('/admin/users');
await authApi.post('/admin/users', { mobileNumber, displayName, role });
await authApi.patch(`/admin/users/${id}`, { isActive: false });
await authApi.delete(`/admin/users/${id}`);
await authApi.get('/admin/sessions', { params: { userId, dateFrom, dateTo } });
```

5.5 Components

`components/AuthContext.jsx`

The global authentication state manager. Provides context consumed by all pages and route guards.

Context value:

```
{
  user: UserObject | null,    // null = unauthenticated
  loading: boolean,           // true during the initial /me check
  checkAuth: () => void,      // re-runs the /me check
  login: (userData) => void,  // called after successful OTP verify
  logout: () => void          // calls POST /logout, then sets user=null
}
```

Initialization: On first mount, calls `GET /api/v1/auth/me`. If the browser cookie is valid, `user` is set from the response. If not (401), `user` stays `null`. This is how the app determines auth state across page refreshes.

`components/ProtectedRoute.jsx`

Role-aware route guard component used by `App.jsx`.

```
Loading state → renders animated spinner (full-screen dark bg)
No user      → <Navigate to="/login" replace />
Wrong role   → <Navigate to="/dashboard" replace />
Authorized   → <Outlet /> (renders the nested child route)
```

Props:

- `allowedRoles: string[]` — e.g., `['admin']` or `['staff', 'admin']`

5.6 Pages

`pages/Home.jsx`

Route: `/`
Access: Public

The corporate public-facing landing page.

Section	Content
Header	Company logo + "S.N. Polymers" brand + conditional nav button (Dashboard if logged in, Login if not)
Hero	"Integrated Digital Business Platform" title with gradient amber text, corporate description
Divisions	Two info cards: Manufacturing Division & Government Infrastructure Projects
Footer	Copyright, audit notice

The "Office Use Log-in" button links to `/login`. If a user is already authenticated, the header shows "Console Dashboard" instead.

`pages/Login.jsx`

Route: `/login`
Access: Public

Mobile number entry form.

Validation & formatting:

```
// Auto-prefix 10-digit Indian numbers
if (/^\d{10}$/.test(formattedNumber)) {
  formattedNumber = `+91${formattedNumber}`;
}
// E.164 validation
if (!/^\+?[1-9]\d{1,14}$/.test(formattedNumber)) { ... }
```

Submit flow:

1. Formats number to E.164.
2. `POST /request-otp` with `{ mobileNumber }`.
3. On success → `navigate('/verify-otp', { state: { mobileNumber } })`.
4. On failure (403) → displays server error message.

UI elements: `+91` static prefix display, amber button, red error box, "Cancel and Return" link.

`pages/OtpVerify.jsx`

Route: `/verify-otp`
Access: Public (redirects to `/login` if no `mobileNumber` in navigation state)

6-digit OTP input with full UX controls.

State variables:

Variable	Purpose
`otp[6]`	Array of 6 digit strings, one per input box
`countdown`	300s (5 min) expiry timer, decrements via `setInterval`
`resendTimer`	30s cooldown before "Request Re-dispatch" appears
`resendDisabled`	Controls button visibility
`loading`, `error`, `success`	Submit state feedback

Input behaviors:

- `onChange` — only accepts digits, auto-focuses next box.
- `onKeyDown` — Backspace on empty box focuses previous box.
- `onPaste` — strips non-digits, distributes 6-digit paste across all inputs.

Submit flow:

1. Joins `otp` array → `"847291"`.
2. `POST /verify-otp` with `{ mobileNumber, otp }`.
3. On success → calls `login(response.data.user)` → shows success message → `navigate('/dashboard')` after 1.2s.
4. On failure → shows error with attempts remaining.

Resend flow:

1. Resets `resendTimer` to 30s, disables button.
2. `POST /request-otp` again.
3. On success → resets countdown to 300s, clears OTP inputs, focuses first box.

`pages/Dashboard.jsx`

Route: `/dashboard`
Access: Staff + Admin (via ProtectedRoute)

Post-login operator console.

Header: Company logo + "Authorized Session" badge + Operator ID + Sign Out button. Admin users see an additional "Access Whitelist Admin" navigation link.

Module cards grid (3 columns):

Card	Title	Status
1	Manufacturing Module	"Phase 2+ Rollout — Access Restricted"
2	Project Management	"Phase 2+ Rollout — Access Restricted"

Card	Title	Status
3	Office Administration	"Active System" — links to `/admin` for admins

The Sign Out button calls `logout()` from `AuthContext`, which hits `POST /logout` and clears the cookie, then sets `user = null` triggering navigation back to `/login`.

`pages/admin/AdminPanel.jsx`

Route: `/admin`
Access: Admin only (via ProtectedRoute)

Full whitelist management interface.

State:

Variable	Purpose
`users[]`	Fetched from `GET /admin/users` on mount
`showAddModal`	Controls "Authorize New Account" modal visibility
`newMobile`, `newName`, `newRole`	Add user form fields
`loading`, `error`, `success`	UI feedback

Data table columns:

- Authorized Account Name
- Authentication Token (Mobile Number — monospace)
- System Privilege Role (badge: indigo=admin, gray=staff)
- Last Verification Access (formatted `toLocaleString()`)
- Verification Count (total session count)
- Firewall Status (toggle button: green=Authorized, red=Deactivated)
- Access Revocation (delete button with confirmation dialog)

Toggle deactivation:

```
const response = await authApi.patch(`/admin/users/${user.id}`, {
  isActive: !user.is_active  // toggles current state
});
```

Add user modal fields: Mobile number, Display name, Role selector (`staff` / `admin`).

`pages/admin/AuditLog.jsx`

Route: `/admin/sessions`
Access: Admin only (via ProtectedRoute)

Full session history ledger with filtering.

State:

Variable	Purpose
`sessions[]`	Fetched from `GET /admin/sessions`
`usersList[]`	Fetched from `GET /admin/users` for filter dropdown
`userIdFilter`	UUID of selected user filter
`dateFrom`, `dateTo`	Date string filters (YYYY-MM-DD)

Filter toolbar: Operator dropdown (populated from all users), Date From picker, Date To picker, Apply + Reset buttons.

Session table columns:

- Operator (display name)
- Verification Token (mobile number — monospace)

- Verification Login Time
- Session Expiry/Logout (shows "Active session" badge in green if `is_active = true`)
- Elapsed Duration (`HH:MM:SS` format, shows "Active Operator" if still open)
- Network Location & Environment (IP address + truncated user-agent string)

Duration formatter:

```
const formatDuration = (seconds) => {
  if (seconds === null || seconds === undefined) return 'Active Operator';
  const hrs  = Math.floor(seconds / 3600).toString().padStart(2, '0');
  const mins = Math.floor((seconds % 3600) / 60).toString().padStart(2, '0');
  const secs = (seconds % 60).toString().padStart(2, '0');
  return `${hrs}:${mins}:${secs}`;
};
```

---

6. Request Lifecycle Walkthroughs

6.1 Full Login Flow

```
User enters mobile number on Login.jsx
  ↓
POST /api/v1/auth/request-otp
  ↓
  [otpRequestLimiter] — max 3/15min per mobile
  ↓
  requestOtp()
    ├─ SELECT authorised_users WHERE mobile_number = ? AND is_active = true
    ├─ 403 if not found
    ├─ crypto.randomInt(100000, 999999) → rawOtp
    ├─ bcrypt.hash(rawOtp, 10) → otpHash
    ├─ INSERT otp_requests (mobile_number, otp_hash, expires_at = +5min)
    └─ Twilio WABA sendMessage (or console.log in dev)
  ↓
200 OK → frontend navigates to /verify-otp

User enters 6-digit OTP on OtpVerify.jsx
  ↓
POST /api/v1/auth/verify-otp
  ↓
  [otpVerifyLimiter] — max 5/5min per mobile
  ↓
  verifyOtpCode()
    ├─ SELECT otp_requests WHERE mobile=? AND is_used=false ORDER BY created_at DESC LIMIT 1
    ├─ Check: now() < expires_at
    ├─ Check: attempts < 3
    ├─ bcrypt.compare(rawOtp, otpHash)
    │   ├─ FAIL: UPDATE attempts++ → 400 with attemptsLeft
    │   └─ PASS: UPDATE is_used=true
    ├─ SELECT authorised_users WHERE mobile_number = ?
    ├─ generateToken(user) → { token, jti }
    ├─ INSERT sessions (user_id, jwt_jti, ip_address, user_agent)
    ├─ res.cookie('token', token, { httpOnly: true, maxAge: 24h })
    └─ notifyAdminLogin(...) ← async, non-blocking
  ↓
200 OK + user data → login(user) → navigate('/dashboard')
```

6.2 Protected Request Flow

```
Any page with verifyJwt middleware:
  ↓
  Browser auto-sends cookie 'token' (withCredentials=true)
  ↓
  verifyJwt()
    ├─ Extract from req.cookies.token
    ├─ jwt.verify(token, JWT_SECRET) → decoded payload
    ├─ SELECT is_active FROM sessions WHERE jwt_jti = decoded.jti
    │   → 401 if missing or is_active=false
    ├─ SELECT is_active FROM authorised_users WHERE id = decoded.user_id
    │   → 403 if missing or is_active=false
    └─ req.user = {...}, req.jti = jti → next()
```

---

7. Security Model

Layer	Control	Implementation
Identity	Mobile number whitelist	`authorised_users` table, checked on every OTP request
Authentication	6-digit time-limited OTP	bcrypt-hashed, 5-min TTL, 3-attempt lockout
Delivery Security	OTP via WhatsApp (end-to-end encrypted)	Twilio WABA API
Token Security	`httpOnly` cookie, never in JS scope	Cookie flags: `httpOnly=true`, `secure=true` (prod), `sameSite=none` (prod)
Token Validation	Live DB session check + JWT signature	`verifyJwt` middleware checks both on every call
Session Invalidation	JTI-based blacklist in DB	`is_active=false` on logout/deactivation blocks token reuse
Admin Segregation	Role check in JWT + DB re-verify	`requireAdmin` middleware, role embedded in token
Rate Limiting	Per-mobile/IP throttling	`express-rate-limit`, OTP: 3/15min, Verify: 5/5min
DB Protection	RLS + Service Role server-side only	Client never touches DB directly
Secret Management	All credentials in `.env`	Production values injected via Render environment dashboard
HTTP Hardening	Security headers	`helmet()` on all responses
CORS	Single allowed origin	`FRONTEND_URL` env var only

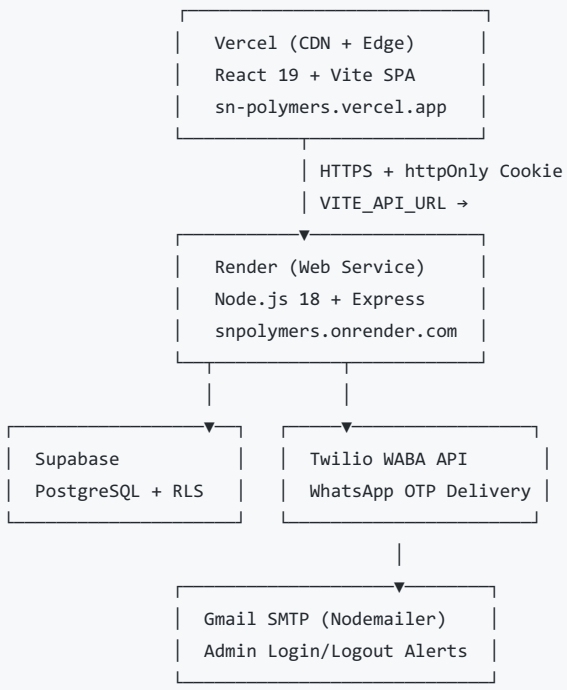
---

8. Deployment Architecture

Live Environments

Service	Provider	URL
Frontend SPA	Vercel	https://sn-polymers.vercel.app/
Backend REST API	Render	https://snpolymers.onrender.com
Database	Supabase	*(private project URL)*

Architecture Diagram



Environment Variable Injection

Variable	Where set
`VITE_API_URL`	Vercel → Project Settings → Environment Variables
All backend `.env` vars	Render → Web Service → Environment

Documentation compiled for S.N. Polymers | Intern Development Batch 2026
Last updated: June 2026